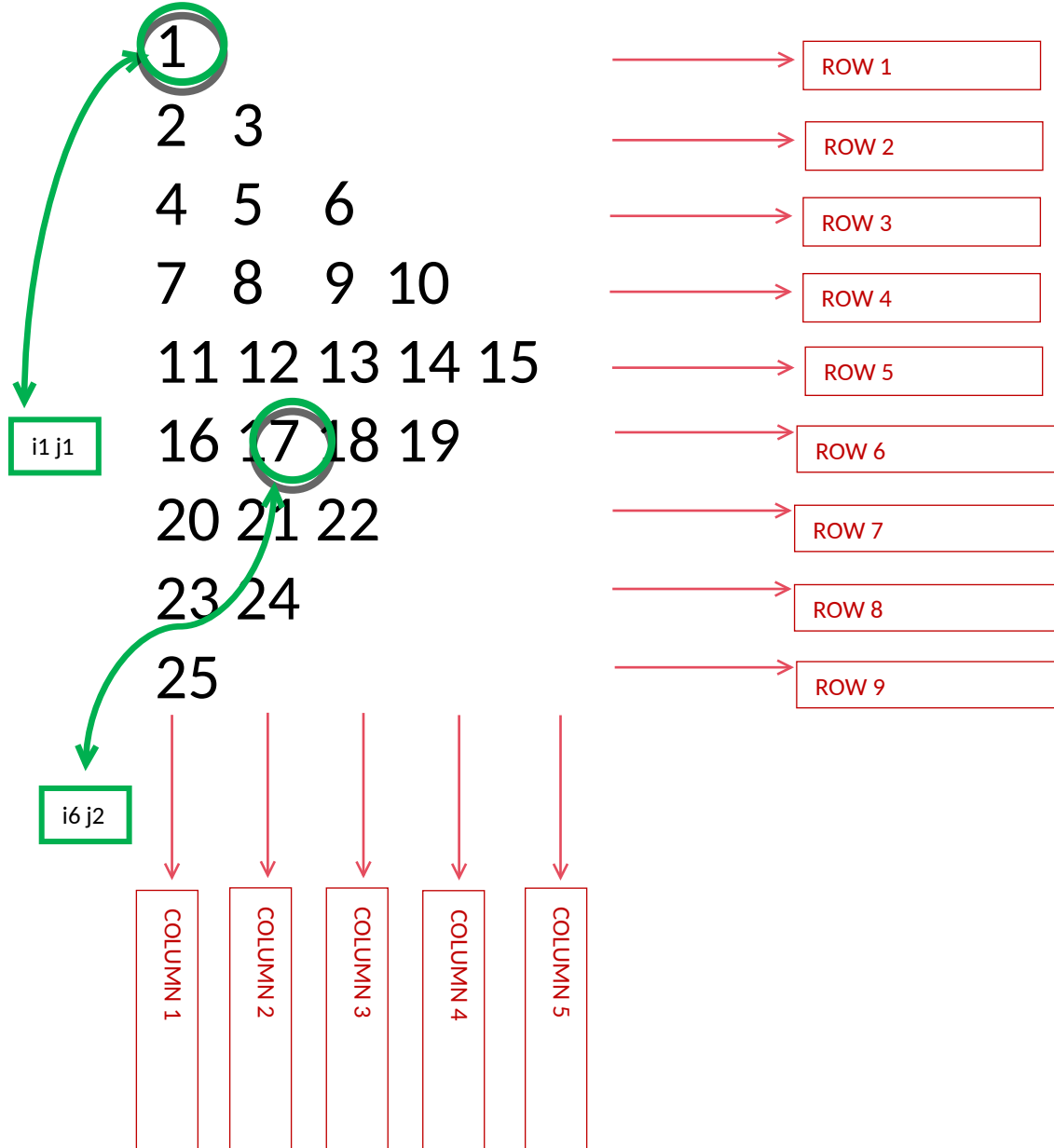


WEEK 01 MODULE
Programming Practice

TOPIC : 2 'Number Pattern'

20. CONTINUOUS NUMBER HALF DIAMOND



1. Problem Analysis :-

i. Pattern Structure:

- The output forms a Continuously increasing number half diamond
- Unlike the Diamond Pattern, this pattern is not centered and does not require leading spaces.

Instead, it consists of two separate parts:

Upper Half Right Angled Triangle + Lower half Inverted Triangle

- The upper half expands downwards
- The lower half contracts downwards

ii. Execution Flow:

Step 1: Observe the Grid Concept:

- For $n = 5$

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
16 17 18 19
20 21 22
23 24
25
```

Upper Half Diamond :-

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

Lower Half Diamond :-

```
16 17 18 19
20 21 22
23 24
25
```

- Each cell in the grid represents a unique coordinate (i, j), where i denotes the row index and j denotes the column index.

-The pattern consist of only one visible region

✦ **Region 1: Numbers**

These form the visible half diamond

For n = 5

Row 0 : [1]

Row 1 : 2 3]

Row 2 : [4 5 6]

Row 3 : [7 8 9 10]

Row 4 : [11 12 13 14 15]

Row 3 : [16 17 18 19]

Row 2 : [20 21 22]

Row 1 : [23 24]

Row 0 : [25]

UPPER HALF :-

- Stars increase by 1 in every row.

LOWER HALF :-

- Stars decrease by 1 in every row.

-> Row Analysis :-

Upper Half :-

Row Index (I)	0	1	2	3	4
numbers	1	2 3	4 5 6	7 8 9 10	11 12 13 14 15

RELATIONSHIP :-

- number = i + 1

Lower Half :-

Row Index (I)	3	2	1	0
Stars	16 17 18 19	20 21 22	23 24	25

RELATIONSHIP :-

- $\text{number} = i + 1$

The formulas remain exactly the same for both halves.

Only the direction of row traversal changes.

Upper Half:

$i = 0 \rightarrow n - 1$

Lower Half:

$i = n - 2 \rightarrow 0$

Step 2: Traverse the grid row by row.

- The outer loop is responsible for generating rows.

Generate the upper half using the Upper Half Right Angled Triangle.

- Traverse rows from 0 to $n-1$.
- Print the required number.

After the upper diamond is completed, generate the lower half.

- Traverse rows from $n-2$ down to 0.
- Print the required number.

Step 3: After all numbers of the current row have been printed, move the cursor to the next line.

Step 4: Repeat the process for all rows until the outer loop terminates.

2. Condition Analysis

No conditional statements are required.

There is no need for `if(...)`

- Every position visited by the star loop prints a number.
- The pattern is controlled entirely through loop limits.

3. Core Logic Transformation

Right Angled Triangle Pattern:-

```
for(int j = 0; j < i + 1; j++)
```

- Meaning:

number increase by 1 in every row.

Inverted Triangle Pattern:-

```
for(int j = 0; j < n - i; j++)
```

- Meaning:

number decrease by 1 in every row.

Half Diamond Pattern:-

```
for(int i = 0; i < n; i++)
```

- Meaning:

Generates the expanding upper triangle.

```
for(int i = n - 2; i >= 0; i--)
```

- Meaning:

Generates the shrinking lower triangle.

The formula for printing numbers remains:

```
for(int j = 1; j <= i + 1; j++)
```

Only the row traversal direction changes.

This creates the complete half diamond shape

4. Program :-

```
package week1_module;
import java.util.Scanner;

public class ContinuousNumberHalfDiamond {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter the number of rows: ");
        int n = sc.nextInt();
        int count=1;
        // Upper Half
        for(int i = 0; i < n; i++) {

            for(int j = 1; j <= i + 1; j++) {
                System.out.print(count+" ");
                count++;
            }

            System.out.println();
        }

        // Lower Half
        for(int i = n - 2; i >= 0; i--) {

            for(int j = 1; j <= i + 1; j++) {
                System.out.print(count+" ");
                count++;
            }

            System.out.println();
        }

        sc.close();
    }
}
```

✦ Why n - 2?

Many students write:

```
for(int i = n - 1; i >= 0; i--)
```

and get:

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
11 12 13 14 15
16 17 18 19
20 21 22
23 24
25
```

here the Middle row is printed twice
as Upper half already printed:

```
11 12 13 14 15
```

So we start from: $i = n - 2$

which skips the duplicate middle row.

hence obtained figure:-

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
16 17 18 19
20 21 22
23 24
25
```

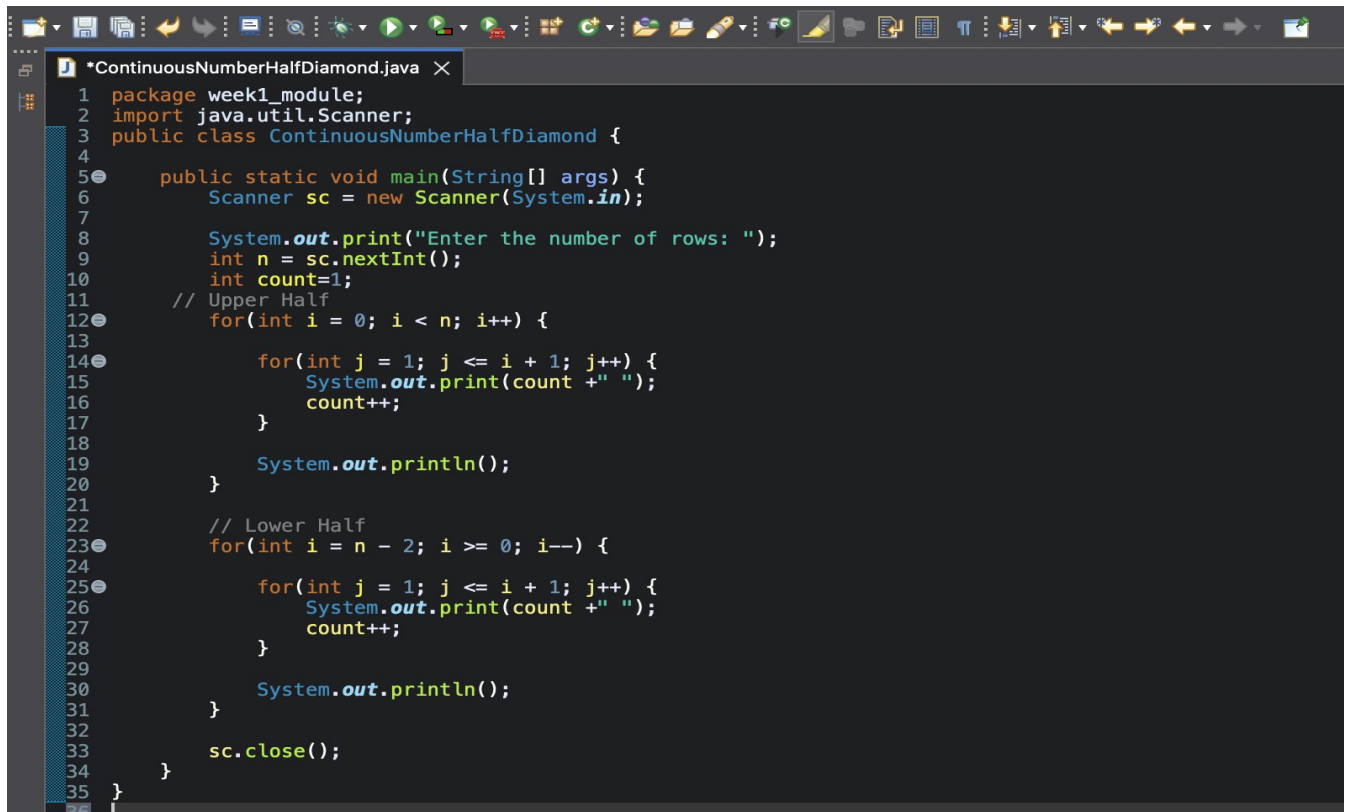
5. Program Output :-

Enter the number of rows: 5

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
16 17 18 19
20 21 22
23 24
```

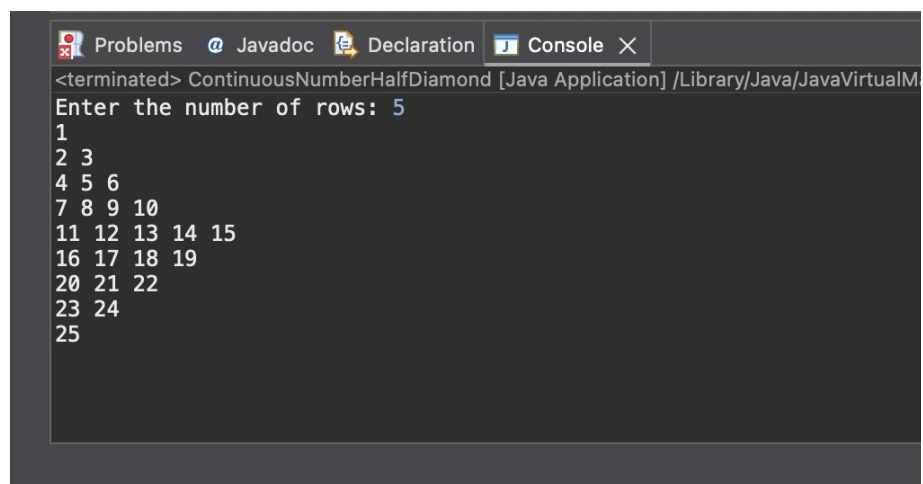
6. ScreenShot :-

i. Program :-



```
1 package week1_module;
2 import java.util.Scanner;
3 public class ContinuousNumberHalfDiamond {
4
5     public static void main(String[] args) {
6         Scanner sc = new Scanner(System.in);
7
8         System.out.print("Enter the number of rows: ");
9         int n = sc.nextInt();
10        int count=1;
11        // Upper Half
12        for(int i = 0; i < n; i++) {
13
14            for(int j = 1; j <= i + 1; j++) {
15                System.out.print(count + " ");
16                count++;
17            }
18
19            System.out.println();
20        }
21
22        // Lower Half
23        for(int i = n - 2; i >= 0; i--) {
24
25            for(int j = 1; j <= i + 1; j++) {
26                System.out.print(count + " ");
27                count++;
28            }
29
30            System.out.println();
31        }
32
33        sc.close();
34    }
35 }
```

ii. Program's Output :-



```
<terminated> ContinuousNumberHalfDiamond [Java Application] /Library/Java/JavaVirtualM...
Enter the number of rows: 5
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
16 17 18 19
20 21 22
23 24
25
```